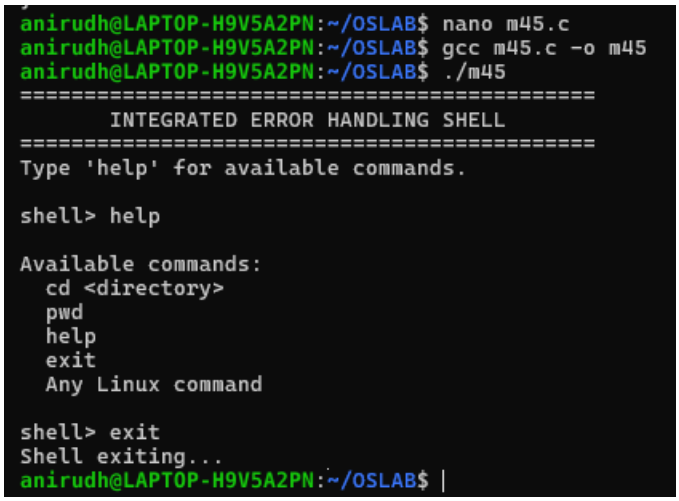

anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ nano m45.c
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ gcc m45.c -o m45
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ ./m45
==========
INTEGRATED ERROR HANDLING SHELL
==========
Type 'help' for available commands.
shell> help
Available commands:
cd <directory>
pwd
help
exit
Any Linux command
shell> exit
Shell exiting...
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$

```
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ cat m45.c
/* integrated_shell_modules.c
 *
 * Connect Modules
 * Validate Interfaces
 * Test End-to-End Execution
 * Handle Errors
 * Improve Reliability
 * Refactor Components
 *
 * Compile:
 * gcc integrated_shell_modules.c -o integrated_shell_modules
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <errno.h>

#define MAX_INPUT 1024
#define MAX_ARGS 64

typedef struct
{
    char *args[MAX_ARGS];
    int count;
} Command;

/* ================ ERROR HANDLING MODULE ================ */

void log_error(const char *message)
{
    fprintf(stderr, "[ERROR] %s: %s\n", message, strerror(errno));
}

void syntax_error(const char *message)
{
    fprintf(stderr, "[SYNTAX ERROR] %s\n", message);
}

void runtime_error(const char *message)
{
    fprintf(stderr, "[RUNTIME ERROR] %s\n", message);
}

/* ================ VALIDATION MODULE ================ */

int validate_input(const char *input)
{
    if (input == NULL || strlen(input) == 0)
    {
        syntax_error("Empty command.");
        return 0;
    }

    if (strstr(input, "||") != NULL)
    {
        syntax_error("Invalid pipe syntax.");
        return 0;
    }

    if (strstr(input, "&&") != NULL)
    {
        syntax_error("Unsupported operator &&.");
        return 0;
    }

    if (strstr(input, ">>") != NULL &&
```

```
anirudh@LAPTOP-N9J5A2PN:~/OSLAB$ cat m45.c
/* integrated_shell_modules.c
 *
 * Connect Modules
 * Validate Interfaces
 * Test End-to-End Execution
 * Handle Errors
 * Improve Reliability
 * Refactor Components
 *
 * Compile:
 * gcc integrated_shell_modules.c -o integrated_shell_modules
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <errno.h>

#define MAX_INPUT 1024
#define MAX_ARGS 64

typedef struct
{
    char *args[MAX_ARGS];
    int count;
} Command;

/* ================= ERROR HANDLING MODULE ================= */

void log_error(const char *message)
{
    fprintf(stderr, "[ERROR] %s: %s\n", message, strerror(errno));
}

void syntax_error(const char *message)
{
    fprintf(stderr, "[SYNTAX ERROR] %s\n", message);
}

void runtime_error(const char *message)
{
    fprintf(stderr, "[RUNTIME ERROR] %s\n", message);
}

/* ================= VALIDATION MODULE ================= */

int validate_input(const char *input)
{
    if (input == NULL || strlen(input) == 0)
    {
        syntax_error("Empty command.");
        return 0;
    }

    if (strstr(input, "||") != NULL)
    {
        syntax_error("Invalid pipe syntax.");
        return 0;
    }

    if (strstr(input, "&&") != NULL)
    {
        syntax_error("Unsupported operator &&.");
        return 0;
    }

    if (strstr(input, ">>") != NULL &&
```

```
        syntax_error("Unsupported operator &&.");
        return 0;
    }

    if (strstr(input, ">>") != NULL &&
        strstr(input, ">") == NULL)
    {
        syntax_error("Invalid redirection syntax.");
        return 0;
    }

    return 1;
}

/* ================ PARSER MODULE ================ */

int parse_command(char *input, Command *cmd)
{
    char *token;

    cmd->count = 0;

    token = strtok(input, " \t\n");

    while (token != NULL)
    {
        if (cmd->count >= MAX_ARGS - 1)
        {
            syntax_error("Too many arguments.");
            return 0;
        }

        cmd->args[cmd->count++] = token;

        token = strtok(NULL, " \t\n");
    }

    cmd->args[cmd->count] = NULL;

    if (cmd->count == 0)
    {
        syntax_error("No command entered.");
        return 0;
    }

    return 1;
}

/* ================ BUILT-IN MODULE ================ */

int execute_builtin(Command *cmd)
{
    if (strcmp(cmd->args[0], "cd") == 0)
    {
        if (cmd->count < 2)
        {
            syntax_error("cd requires a directory.");
            return 1;
        }

        if (chdir(cmd->args[1]) == -1)
        {
            log_error("cd failed");
        }

        return 1;
    }

    if (strcmp(cmd->args[0], "pwd") == 0)
    {
        char cwd[1024];
```

```
    }

    if (strcmp(cmd->args[0], "pwd") == 0)
    {
        char cwd[1024];

        if (getcwd(cwd, sizeof(cwd)) == NULL)
        {
            log_error("Unable to get current directory");
        }
        else
        {
            printf("%s\n", cwd);
        }

        return 1;
    }

    if (strcmp(cmd->args[0], "exit") == 0)
    {
        printf("Shell exiting...\n");
        exit(0);
    }

    if (strcmp(cmd->args[0], "help") == 0)
    {
        printf("\nAvailable commands:\n");
        printf("  cd <directory>\n");
        printf("  pwd\n");
        printf("  help\n");
        printf("  exit\n");
        printf("  Any Linux command\n");

        return 1;
    }

    return 0;
}

/* ================ EXECUTION MODULE ================ */

int execute_external(Command *cmd)
{
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0)
    {
        log_error("fork failed");
        return -1;
    }

    if (pid == 0)
    {
        execvp(cmd->args[0], cmd->args);

        fprintf(
            stderr,
            "[EXECUTION ERROR] Command '%s' not found or cannot execute: %s\n",
            cmd->args[0],
            strerror(errno)
        );

        exit(127);
    }

    if (waitpid(pid, &status, 0) == -1)
    {
        log_error("waitpid failed");
```

```c
/* ================ MAIN SHELL MODULE ================ */

int main()
{
    char input[MAX_INPUT];

    printf("===========================================\n");
    printf("       INTEGRATED ERROR HANDLING SHELL\n");
    printf("===========================================\n");

    printf("Type 'help' for available commands.\n");

    while (1)
    {
        printf("\nshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            if (feof(stdin))
            {
                printf("\nEOF received. Exiting.\n");
                break;
            }

            log_error("Input error");
            clearerr(stdin);
            continue;
        }

        input[strcspn(input, "\n")] = '\0';

        if (!validate_input(input))
        {
            printf("Recovering from syntax error...\n");
            continue;
        }

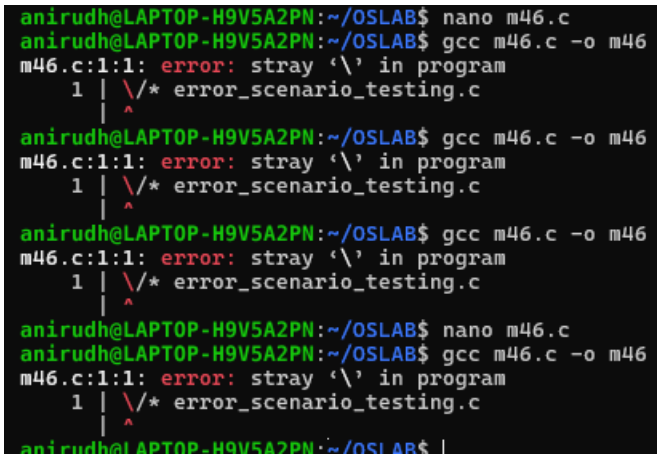
        Command cmd;

        if (!parse_command(input, &cmd))
        {
            printf("Recovering from parsing error...\n");
            continue;
        }

        if (execute_builtin(&cmd))
        {
            continue;
        }

        if (execute_external(&cmd) != 0)
        {
            printf("Shell recovered and is ready for another command.\n");
        }
    }

    return 0;
}
```


anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ nano m46.c
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ gcc m46.c -o m46
m46.c:1:1: error: stray '\' in program
1 | \/* error_scenario_testing.c
| ^
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ gcc m46.c -o m46
m46.c:1:1: error: stray '\' in program
1 | \/* error_scenario_testing.c
| ^
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ gcc m46.c -o m46
m46.c:1:1: error: stray '\' in program
1 | \/* error_scenario_testing.c
| ^
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ nano m46.c
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ gcc m46.c -o m46
m46.c:1:1: error: stray '\' in program
1 | \/* error_scenario_testing.c
| ^

```
\/* error_scenario_testing.c
 *
 * Detect Runtime Errors
 * Generate Messages
 * Log Failures
 * Handle Invalid Syntax
 * Recover Gracefully
 * Test Error Scenarios
 *
 * Compile:
 * gcc error_scenario_testing.c -o error_scenario_testing
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>
#include <sys/wait.h>

void log_failure(const char *operation)
{
    fprintf(
        stderr,
        "[LOG] FAILURE: %s -> %s\n",
        operation,
        strerror(errno)
    );
}

void test_invalid_command()
{
    pid_t pid;
    int status;

    printf("\n==========================================\n");
    printf("TEST 1: INVALID COMMAND\n");
    printf("==========================================\n");

    pid = fork();

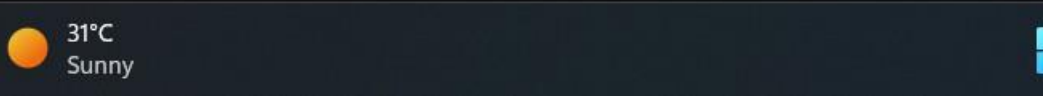
    if (pid < 0)
    {
        log_failure("fork");
        return;
    }

    if (pid == 0)
    {
        execlp(
            "command_that_does_not_exist",
            "command_that_does_not_exist",
            NULL
        );

        fprintf(
            stderr,
            "[ERROR] Invalid command detected: %s\n",
            strerror(errno)
        );

        exit(127);
    }

    waitpid(pid, &status, 0);
```



```
        );

        printf("Redirection error handled successfully.\n");
    }
}

void test_permission_error()
{
    int fd;

    printf("\n===========================================\n");
    printf("TEST 5: FILE ACCESS ERROR\n");
    printf("===========================================\n");

    fd = open(
        "/root/restricted_file.txt",
        O_WRONLY
    );

    if (fd == -1)
    {
        log_failure("Accessing restricted file");
        printf("Permission error handled gracefully.\n");
        return;
    }

    close(fd);
}

void test_recovery()
{
    printf("\n===========================================\n");
    printf("TEST 6: ERROR RECOVERY\n");
    printf("===========================================\n");

    printf("Simulating runtime failure...\n");

    errno = ENOMEM;

    fprintf(
        stderr,
        "[RUNTIME ERROR] Memory allocation failed: %s\n",
        strerror(errno)
    );

    printf("Cleaning temporary state...\n");
    printf("Restoring shell state...\n");
    printf("Continuing normal execution...\n");

    printf("Recovery successful.\n");
}

int main()
{
    printf("===========================================\n");
    printf("          ERROR SCENARIO TEST SUITE\n");
    printf("===========================================\n");

    test_invalid_command();
    test_invalid_file();
    test_invalid_syntax();
    test_invalid_redirection();
    test_permission_error();
    test_recovery();

    printf("\n===========================================\n");
    printf("           ALL TESTS COMPLETED\n");
    printf("===========================================\n");

    return 0;
}
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ |
```